

Urban crash analytics: a spatial analysis of road collision risk in New York City

Saif Ullah Arshad (BSCS23065)

Session: Spring 2026

Supervisor: Dr. Adnan Siddique

Course: Spatial Data Science

1. Introduction

Cities usually fix dangerous roads only after many accidents have already happened - a speed bump gets added, a sign goes up, but people have already been hurt. This project asks: can we predict which roads are high risk before accidents occur?

We analyze **500,000 crash records** from **NYC (2021–2026)**, spatially join them to 122,272 road segments using GeoPandas, compute a Collision Severity Index per segment, and apply spatial techniques including **Moran's I, LISA, KDE**, and spatial regression to understand crash patterns. We then train an XGBoost classifier achieving $F1 = 0.848$ to predict collision risk class for every road in the city.

2. Problem Statement

Can we predict which road segments in NYC are at high collision risk - before accidents happen - using only road infrastructure features and spatial analysis?

Authorities currently react after repeated incidents. We propose a proactive approach: identify danger using road design so action happens before people get hurt.

3. Data

Dataset 1 - NYC Motor Vehicle Collisions (NYC Open Data, Socrata API)

- 500,000 crash records, February 2021 to March 2026
- Fields: GPS coordinates, date, time, injuries, fatalities

Dataset 2 - NYC Street Centerline (NYC Open Data, GeoJSON API)

- 122,272 road segments covering all five boroughs
- Fields: speed limit, lane count, road width, traffic direction

Preprocessing:

- Dropped 38,575 rows with missing GPS (7.7%)
- Parsed datetime, removed redundant columns
- Computed Collision Severity Index per crash:
 $CSI = 1 + (2 \times \text{injuries}) + (5 \times \text{fatalities})$
- Aggregated CSI per road segment $\rightarrow$ Low / Moderate / High risk

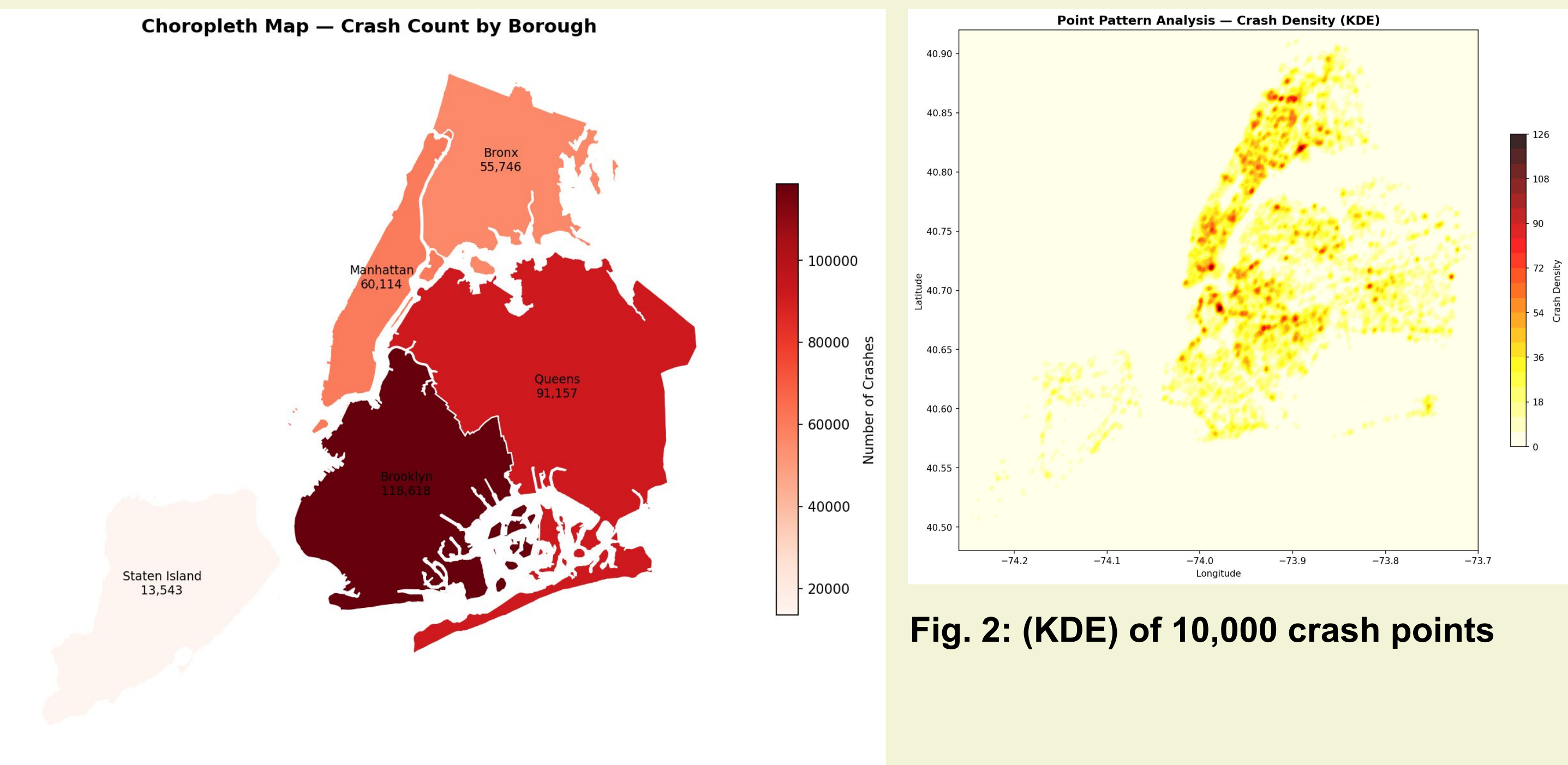


Fig. 1: Choropleth map showing crash count per NYC borough. Brooklyn and Queens record the highest crash frequency.

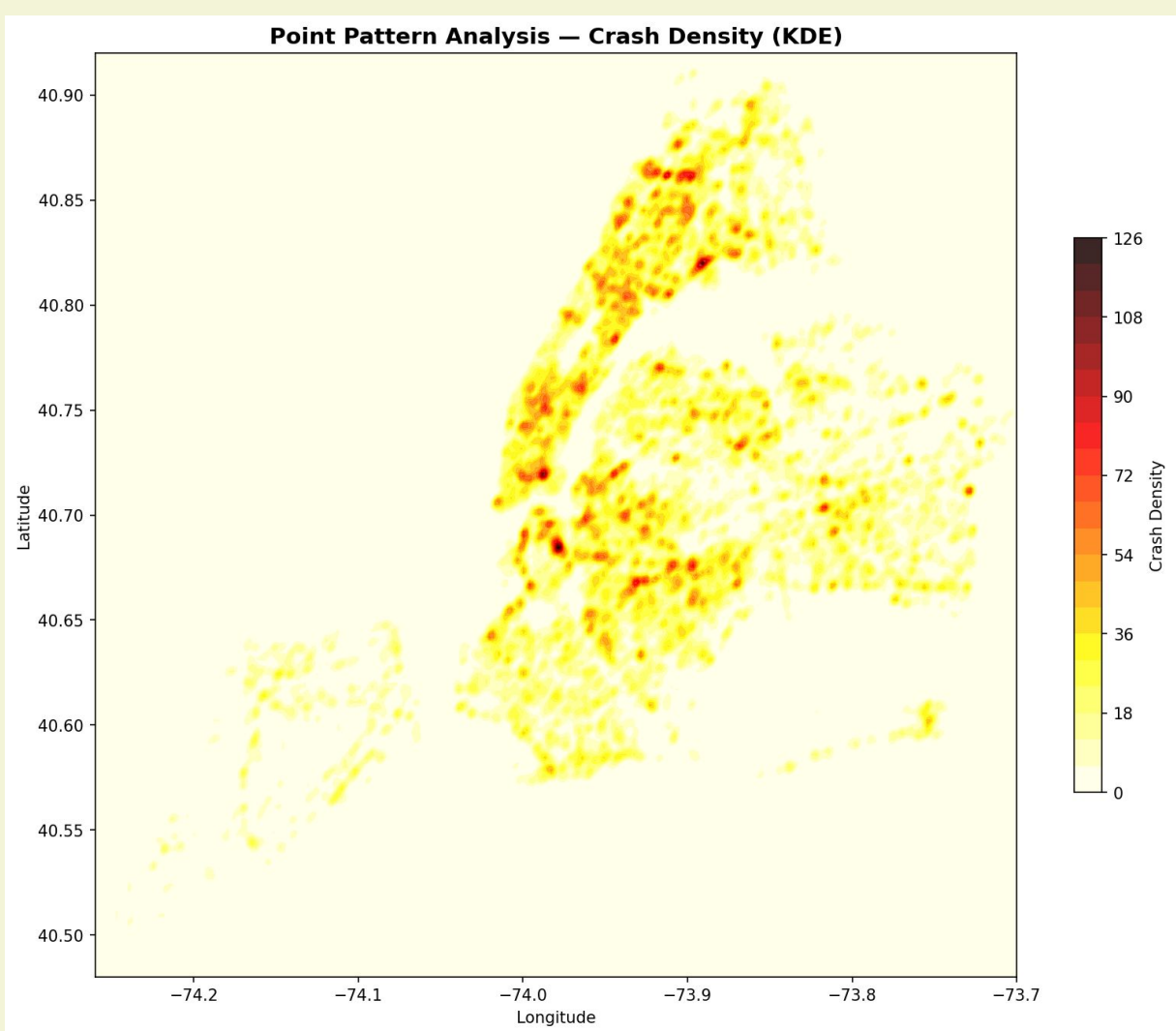


Fig. 2: (KDE) of 10,000 crash points

4. Methodology

- Step 1 - **Spatial Join**
 - Each crash GPS point snapped to nearest road segment using GeoPandas
 - `sjoin_nearest()` in EPSG:3857 (metric), max distance 50m.
 - 461,425 crashes connected to 63,265 road segments.
- Step 2 - **Choropleth Mapping**
 - Crash count and risk class visualized across boroughs and road segments
 - using color-coded maps.
- Step 3 - **Spatial Autocorrelation**
 - Global Moran's I and LISA computed using libpysal Queen contiguity
 - weights to identify clustering and hotspot locations.
- Step 4 - **Point Pattern Analysis**
 - Kernel Density Estimation (KDE) applied to 10,000 crash points to
 - produce a continuous crash density surface.
- Step 5 - **Spatial Regression**
 - OLS vs Spatial Lag Model compared to confirm spatial dependency
 - in crash risk across neighboring road segments.
- Step 6 - **ML Classification**
 - XGBoost and Random Forest trained on road features. SMOTE used to
 - handle class imbalance. Evaluated with F1 and ROC-AUC per borough.

5. Results & Discussion

Spatial Analysis:

- Moran's I = 0.164, $p = 0.001 \rightarrow$ high risk roads cluster geographically
- LISA: 33 High-High hotspots, 2,602 Low-Low coldspots identified
- KDE: crash density peaks in Manhattan and inner Brooklyn/Queens
- Spatial Regression: R^2 improved from 0.5674 to 0.5691 with spatial lag
Positive coefficient confirms neighboring risk predicts segment risk

Machine Learning:

- XGBoost: $F1 = 0.848$, $AUC = 0.971$ (best model)
- Random Forest: $F1 = 0.840$, $AUC = 0.961$
- Overall accuracy: 89%
- Borough $F1$: 0.830 to 0.852 - consistent across all five boroughs
- Most important feature: crash density (importance = 0.57)
- Future work: add weather features, real-time traffic data,
- extend to other cities, deploy as public web application.

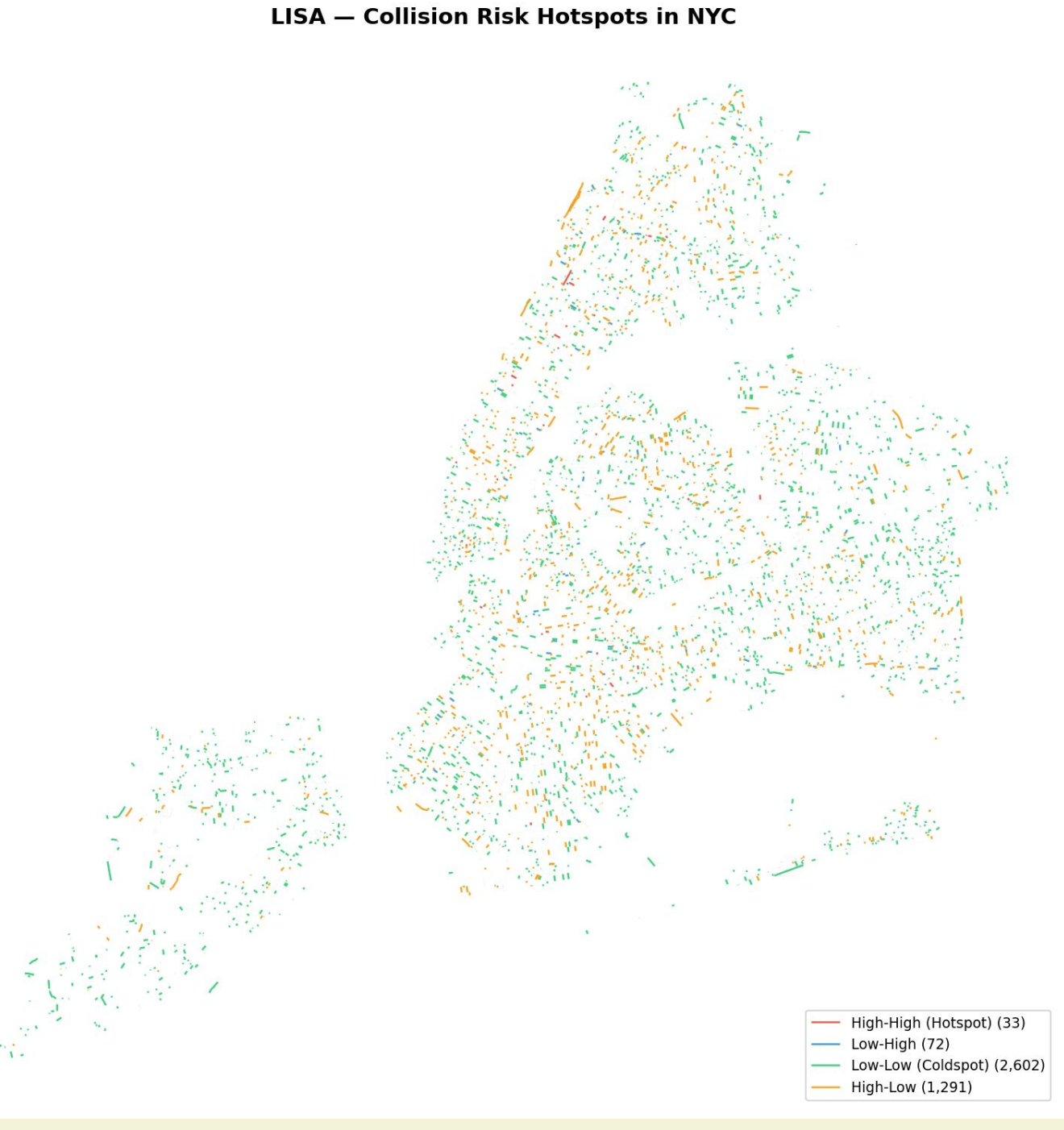


Fig. 3: LISA hotspots (Moran's I = 0.164, $p=0.001$)

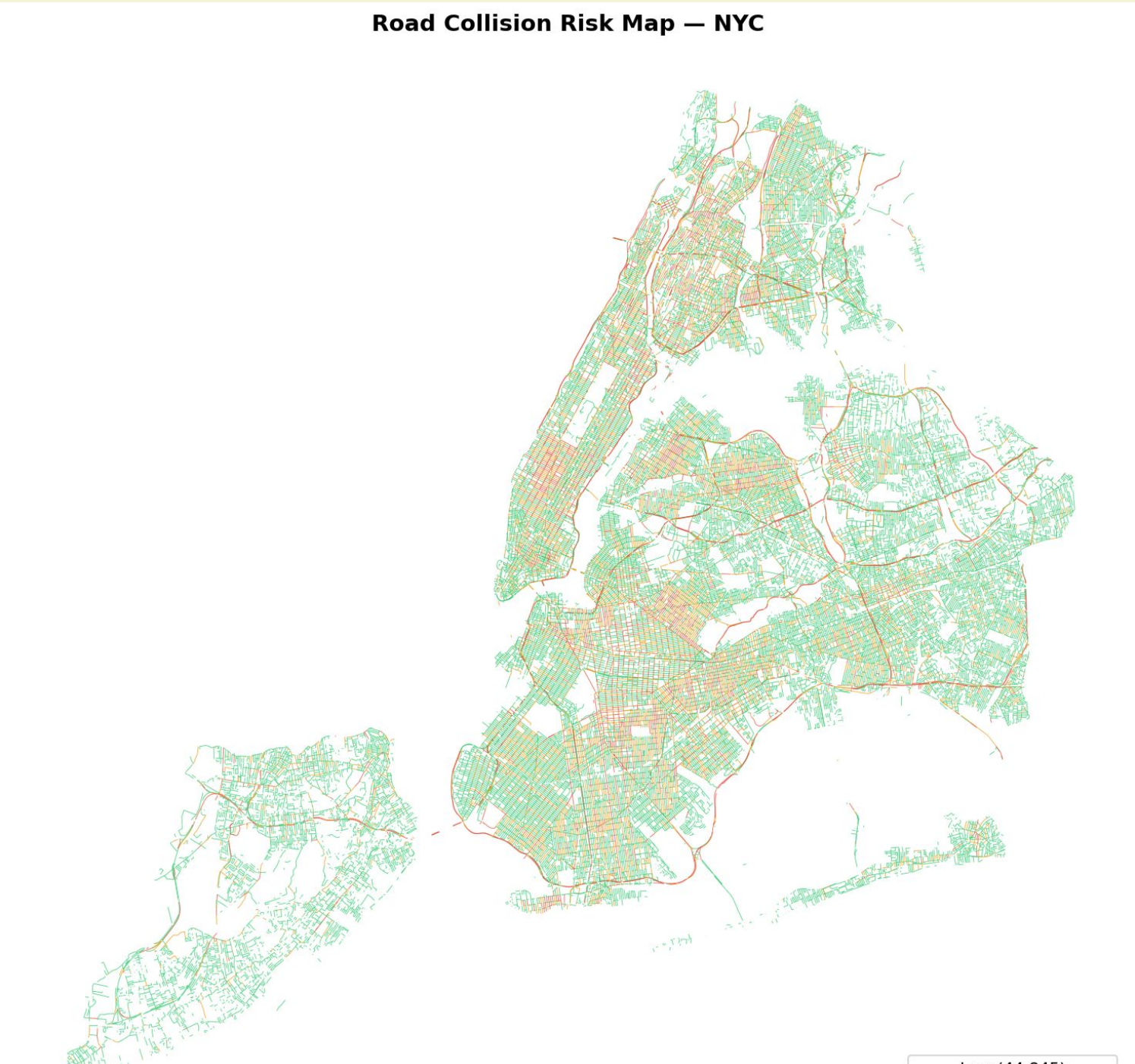


Fig. 4: Road collision risk map

Acknowledgments: Please add acknowledgements here. Data: NYC Open Data, NYC Department of City Planning Libraries: GeoPandas, libpysal, esda, Scikit-Learn, XGBoost, Folium